

Overall architecture:

The system accelerates two components of a genre classification pipeline on an FPGA: a radix-2 FFT preprocessor and a 2-layer LSTM inference engine. The full pipeline is:

None

```
Raw audio → FFT (Spatial) → log magnitude (Python) → normalize  
(Python)  
→ proj 512→128 (Python) → LSTM + FC (Spatial) → genre index
```

The CNN from the original model was removed because its input weight matrix (512×4096) was too large for on-chip storage. Instead the FFT output is used directly as the frequency feature, with a lightweight linear projection running in software.

Eva FFT design choices:

For the FFT, we use a radix-2 Cooley Tookey FFT. It's a classic, simpler algorithm that numpy and scipy use. We use 32 fixed point with 8 integer bits and 24 fractional bits since it's less expensive in hardware and fine precision, which is important for FFTs especially with smaller, normalized integers. Some of the looping, like the stage loop, is sequential because of the sequential in nature FFT algorithm. Lastly, we use a windowing function before the FFT to reduce spectral leakage.

Eva FFT optimization choices:

There were a number of optimizations for the FFT portion of our project. Firstly, there is a lot of host side computation that limits expensive operations in hardware, such as sin and cosine. We stored the constants in LUTs, which the accelerator can quickly access them. A second optimization is using preallocation buffers in SRAM for the real and imaginary components of the FFT. That is, the design alternates between reading from a set of buffers and writing outputs to another set before copying results back for the next stage. The last notable optimization is batched frame processing which lets multiple audio frames be processed at once, limiting overhead of starting and increasing throughput.

Sara LSTM design choices:

The LSTM implements a standard 2-layer LSTM cell with hidden size 128, processing 50 timesteps sequentially. All weight matrices are loaded from DRAM into on-chip SRAMs at

the start. This avoids repeated DRAM accesses during the sequential timestep loop, trading BRAM usage for latency. **Matrix-vector multiply:** For each timestep and each gate, the input contribution and hidden state contribution are computed as separate dot products using **Reduce** over the input/hidden dimension. These are added together with the bias terms to produce the pre-activation gate value. **Sigmoid and tanh approximation:** We realised that hardware cant natively compute e^x . Instead sigmoid is implemented via a 256-entry LUT covering the range $[-6, 6]$: outside this range sigmoid saturates to 0 or 1. The input is clipped using mux(hardware multiplexer) and mapped to a LUT index via linear interpolation. Tanh is derived from sigmoid using the identity $\tanh(x) = 2 \cdot \text{sigmoid}(2x) - 1$, avoiding a separate LUT. **Fixed point type:** `FixPt[TRUE, _16, _16]` (32 bits, signed) gives 16 bits integer and 16 bits fractional precision, sufficient for LSTM weights. **FC layer** performs a simple 128 to 10 matrix-vector multiply on the final hidden state h_1 , producing 10 logits. Argmax selects the predicted genre index.

Sara optimization choices:

All LSTM weight matrices are preloaded from DRAM into on-chip SRAM once before the timestep loop, so all 50 timesteps access fast on-chip memory instead of slower off-chip DRAM. Sigmoid is implemented via a 256-entry LUT precomputed at compile time rather than CORDIC or Taylor series, with tanh reusing the same LUT via $\tanh(x) = 2 \cdot \text{sigmoid}(2x) - 1$. Weights are quantized to 32-bit fixed point instead of float32, reducing multiplier cost while maintaining 66.5% test accuracy. Finally, the original CNN was removed since its 512×4096 weight matrix was too large for on-chip storage. FFT features are instead fed directly into the LSTM with a lightweight software projection step